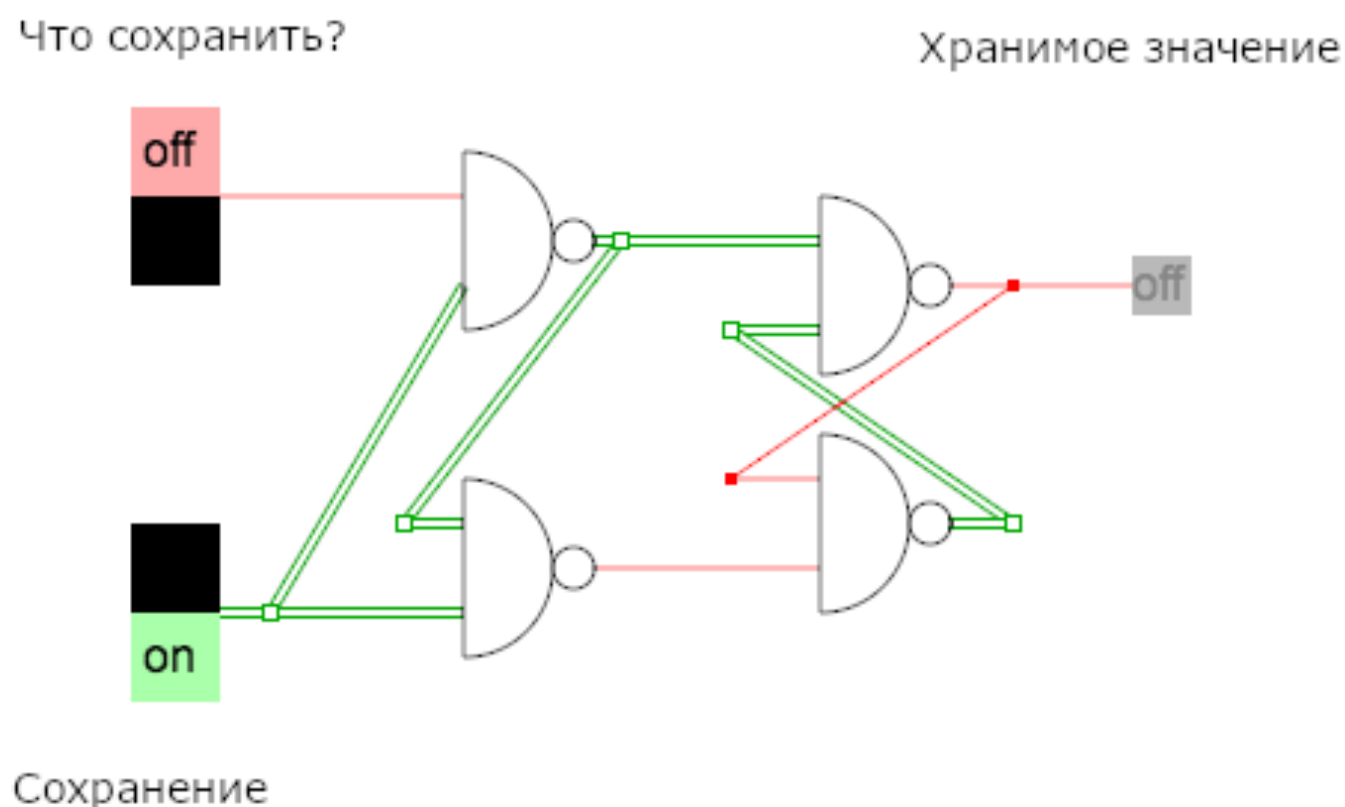
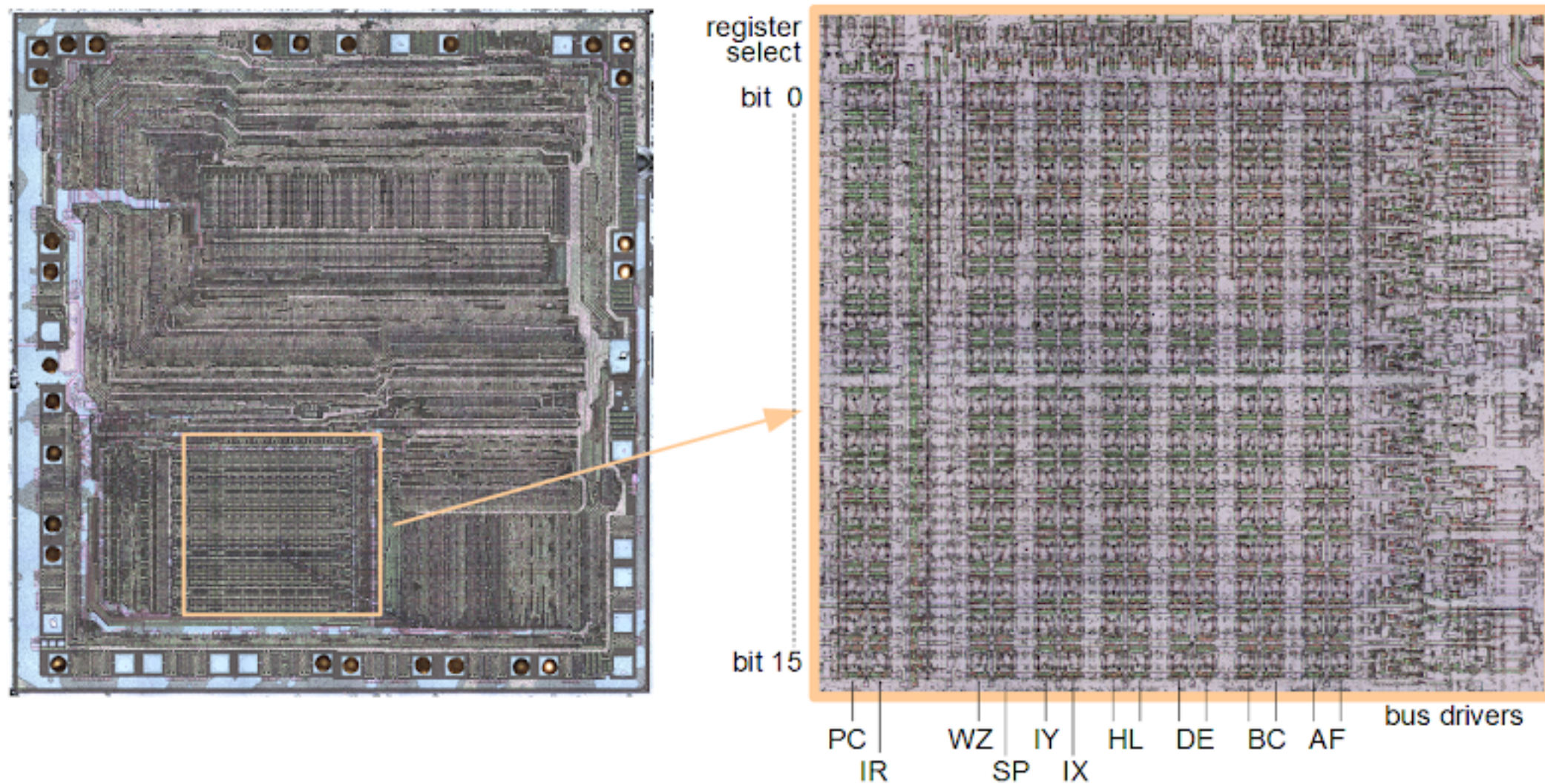


Иерархия памяти

Современные CPU имеют достаточно сложное устройство, которое включает в себя быструю память в виде регистров, нескольких уровней кеш памяти, жёсткий диск или ssd накопитель. Самой быстрой памятью являются регистры. Один регистр это комбинация схем, каждая из которых хранит один бит, каждая схема называется триггером или в англоязычной литературе существует термин D-Latch. Схема состоит из транзисторов и способна сохранять заряд. На сайте <http://simplecpu.com/memory.html> есть анимированная модель триггера на NAND элементах.

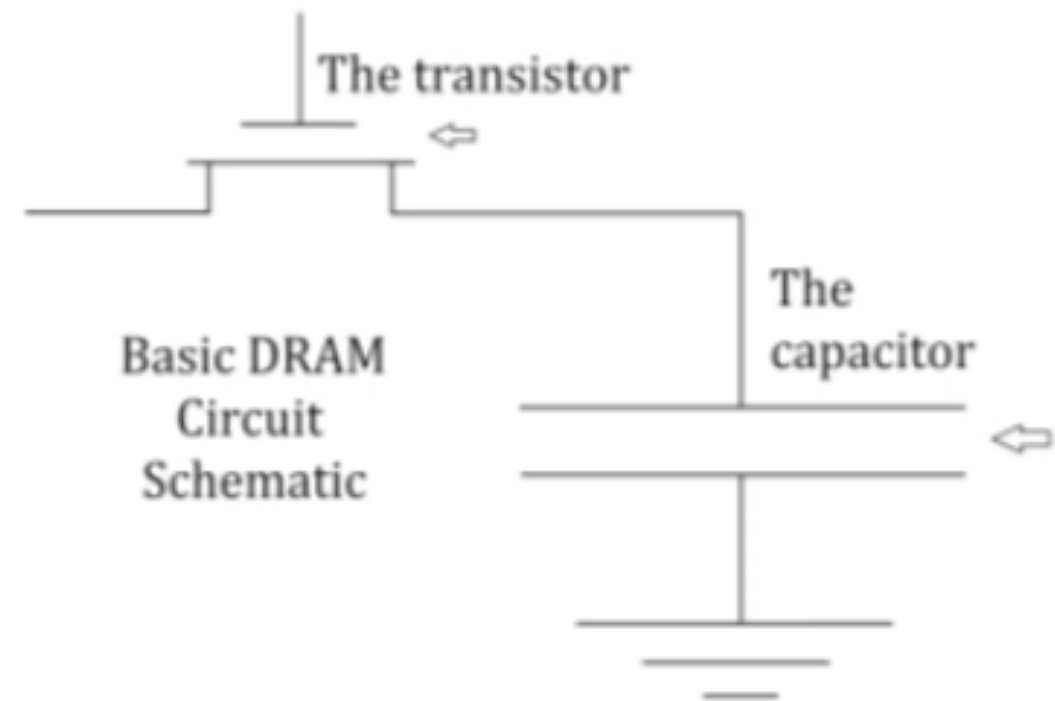
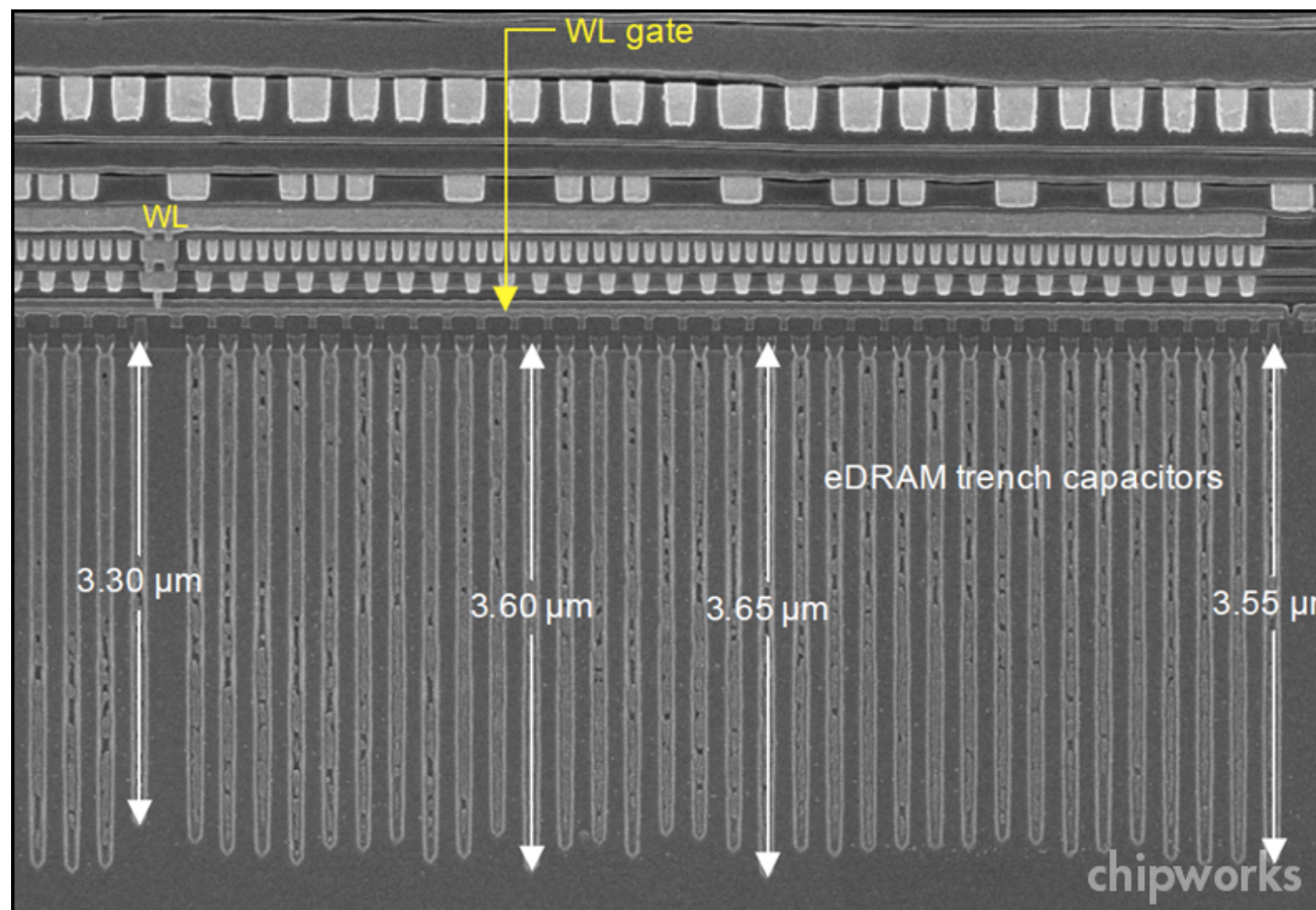


Тип памяти, используемый для регистров называется static random access memory (SRAM). Фотография массива регистров в реальном процессоре Z80.



В отличие от ячейки статической памяти, элемент динамической памяти (DRAM) состоит из всего двух частей — одного транзистора и одного конденсатора.

Фотография встроенной DRAM процессора IBM Power 7+:



Плюсы SRAM:

- высокая скорость зарядки/разрядки ячейки

Минусы SRAM:

- относительная сложность изготовления

Плюсы DRAM:

- относительная простота изготовления / дешевизна

Минусы DRAM:

- низкая скорость зарядки/разрядки ячейки

Из SRAM ячеек как правило состоят регистры и кеш, из DRAM ячеек оперативная память.

Порядки задержек по времени, характерных для некоторых процессов:

Latency Comparison Numbers (~2012)

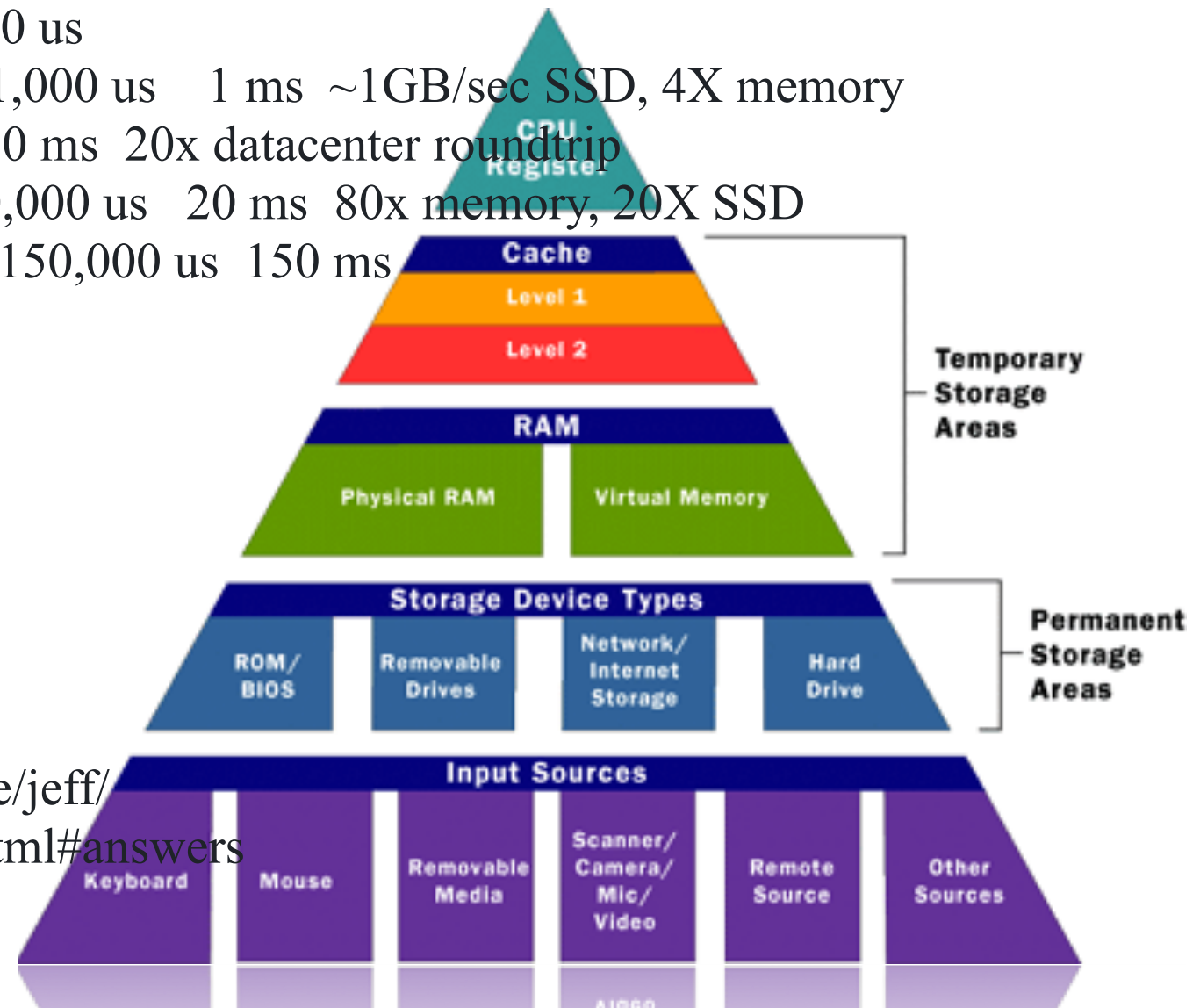
L1 cache reference	0.5 ns			
Branch mispredict	5 ns			
L2 cache reference	7 ns	14x L1 cache		
Mutex lock/unlock	25 ns			
Main memory reference	100 ns	20x L2 cache, 200x L1 cache		
Compress 1K bytes with Zippy	3,000 ns	3 us		
Send 1K bytes over 1 Gbps network	10,000 ns	10 us		
Read 4K randomly from SSD*	150,000 ns	150 us	~1GB/sec SSD	
Read 1 MB sequentially from memory	250,000 ns	250 us		
Round trip within same datacenter	500,000 ns	500 us		
Read 1 MB sequentially from SSD*	1,000,000 ns	1,000 us	1 ms	~1GB/sec SSD, 4X memory
Disk seek	10,000,000 ns	10,000 us	10 ms	20x datacenter roundtrip
Read 1 MB sequentially from disk	20,000,000 ns	20,000 us	20 ms	80x memory, 20X SSD
Send packet CA->Netherlands->CA	150,000,000 ns	150,000 us	150 ms	

Notes

1 ns = 10^-9 seconds
1 us = 10^-6 seconds = 1,000 ns
1 ms = 10^-3 seconds = 1,000 us = 1,000,000 ns

Credit

By Jeff Dean: <http://research.google.com/people/jeff/>
Originally by Peter Norvig: <http://norvig.com/21-days.html#answers>



Обычно каждое ядро имеет свой собственный кеш верхнего уровня (обычно это кеши первого и второго уровня) и один общий на третьем уровне (уровней бывает больше). Кеш верхнего уровня обычно делится на кеш инструкций и кеш данных (i и d соответственно). Такой подход не очень соответствует архитектуре фон Неймана. В начале, когда кеши только появились этого разделения не было, однако быстро заметили, что инструкции и данные имеют слишком разный характер доступа. Например инструкции меняются достаточно редко, поэтому кеш инструкций в значительной мере позволяет ускорить доступ благодаря пространственной локальности, однако с данными так работать не всегда получается, так как они часто генерируются по мере выполнения программы.

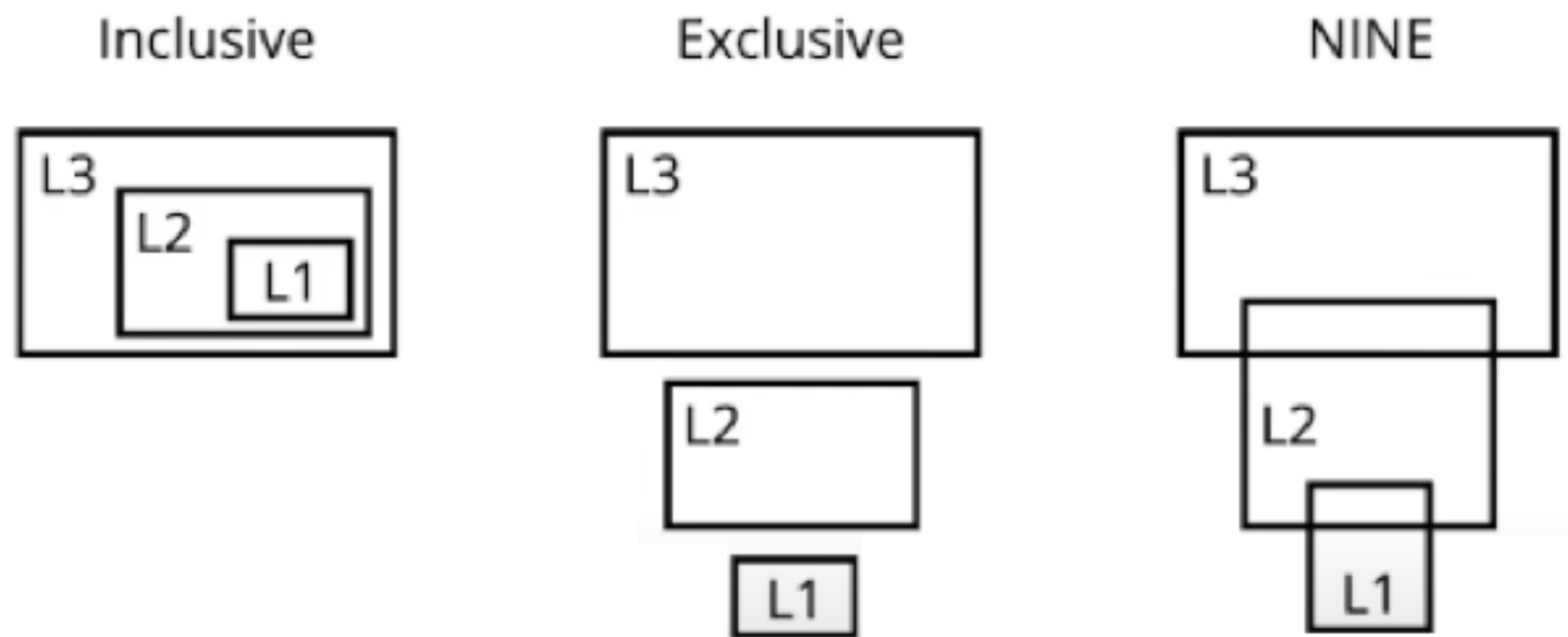
Кеши нижних уровней также состоят из таких же SRAM ячеек памяти, обычно имеют больший параметр latency в связи с тем, что они физически дальше удалены от процессора. Также еще играет роль поиск данных в кеше — чем больше размер памяти, тем сложнее в ней искать нужный фрагмент. Зачастую поиск в кеше линейный, однако благодаря тому, что он реализован аппаратно latency увеличивается не сильно, однако он гораздо больше чем в кеше верхнего уровня.

Так как имеется несколько уровней, то необходимо понимать, как данные распределяются по этим уровням. За это распределение отвечают специальные политики.

Inclusive — все уровни кеша хранят одни и те же данные, кеши верхних уровней хранят наиболее актуальные кеш строки.

Exclusive — все уровни кеша хранят разные данные, это позволяет увеличить количество закешированных данных, но реализуется достаточно сложно.

NINE (Not Inclusive Not Exclusive) — гибридный, часть данных дублируется от уровня к уровню.



В режиме Inclusive если в кеше верхнего уровня уже лежат нужные данные, то они будут взяты из него. Если в кеше верхнего уровня уже есть некоторые посторонние данные, а в кеше нижнего есть требуемые данные, то требуемые данные сначала будут скопированы в кеш верхнего уровня, в случае если места будет не хватать то требуемые данные могут вытеснить посторонние данные в кеше верхнего уровня. Если в кешах верхнего уровня и нижнего нет нужных данных, то происходит чтение в эти оба кеша.

В режиме Exclusive если в кеше верхнего уровня уже лежат нужные данные, то они будут взяты из него. Если в кеше верхнего уровня нет необходимых данных, а в кеше нижнего уровня они есть, то данные будут скопированы из кеша нижнего уровня в кеш верхнего, а далее инвалидированы в кеше нижнего уровня. Если данных нет в кеше нижнего и верхнего уровня, то данные будут прочитаны только в кеш верхнего уровня. Записи в книжных кешах в таком режиме появляются из-за вытеснения из верхнего уровня. То есть в кешах верхнего уровня лежат «новые данные», в кешах нижнего уровня остаются «старые данные».

В режиме NINE если в кеше верхнего уровня уже лежат нужные данные, то они будут взяты из него. Если в кеше верхнего уровня отсутствуют требуемые данные, а в кеше нижнего уровня они присутствуют то происходит тоже самое, что и в режиме Inclusive. Если в нижнем и верхнем уровне нет требуемых данных, то подобно режиму Exclusive чтение происходит в кеш верхнего уровня.

Соответственно процессор может осуществлять операции только над данными, которые имеются в кеше. Данные попадают в кеш специальными порциями — кеш линиями. Обычно кеш линия имеет размер 64 байта. Процессор, оперируя непрерывным адресным пространством, побуждает алгоритмы, доставляющие кеш линии, выбирать нужные данные и копировать их в кеш. Даже если нужны данные размером меньше 64 байт, а занимающие например 1 байт, то будет скопирован этот байт и ещё 63 соседних. Сложное устройство современных CPU предполагает наличие модуля, помогающего предугадать следующие операции с памятью и позволяющие заранее вычитывать эту память в кеш.

Процессор может запросить данные, лежащие в «предсказуемых» областях (пространственная локальность) и они будут своевременно доставлены в кеш из оперативной памяти. Обращение к ячейке памяти статистически увеличивает вероятность повторного с ней взаимодействия (временная локальность). Или напротив, данные могут загружаться из случайных адресов (например деревья и связанные списки) и это может вызвать трудности с предсказанием следующих адресов, что в свою очередь влечёт ситуацию при которой запрошенный процессором адрес не будет в наличие в текущий момент времени. Такая ситуация называется кеш промахом (или cache miss). Кроме того промах может быть вызван конечностью размера кеша по сравнению с оперативной памяти, которая может быть больше на порядки.

Кеш промахи делятся на несколько видов:

Промах по чтению из кэша инструкций обычно дает очень большую задержку, поскольку процессор не может продолжать исполнение программы (по крайней мере, текущего потока исполнения) и вынужден простаивать в ожидании загрузки инструкции из памяти.

Промах по чтению из кэша данных дает меньшую задержку, поскольку инструкции, не зависящие от запрошенных данных, могут продолжать исполняться, пока запрос обрабатывается в основной памяти. После получения данных из памяти можно продолжать исполнение зависимых инструкций.

Промах по записи в кэш данных дает наименьшую задержку, поскольку запись может быть поставлена в очередь и последующие инструкции практически не ограничены в своих возможностях. Процессор может продолжать свою работу, кроме случаев промаха по записи с полностью заполненной очередью. Если данные уже присутствуют в кеше, то это благотворно сказывается на скорости исполнения программы и называется Кеш попаданием (`cache hit`).

Прямое управление кешем не осуществляется программистом, а происходит на автоматизированном уровне. Однако можно воздействовать на кеш косвенными методами, такими как:

Выравнивание данных: Выравнивание данных по границам кэш-линий может улучшить эффективность использования кэша. Это можно достичь, например, с помощью ключевых слов `alignas` или `__attribute__((aligned))` в C/C++ для выравнивания переменных или структур данных.

Локализация данных: Помимо обеспечения последовательного доступа к данным, можно стараться организовывать данные таким образом, чтобы данные, которые часто используются вместе, располагались близко друг к другу в памяти. Это может уменьшить промахи кэша и увеличить вероятность попадания данных в кэш.

Использование кэш-предварительного прогрева (Cache Pre-Warming): Прогрев кэша — это процесс загрузки данных в кэш перед началом активного использования. Программист может явно выполнить чтение или запись в области памяти, чтобы активировать загрузку данных в кэш и снизить промахи кэша при дальнейших операциях. В компиляторах, например в `gcc` и `clang` существуют встроенные функции для прогрева кэша: `void __builtin_prefetch(const void *addr, int rw, int locality);`, где `addr` это указатель на предзагружаемые данные, `rw` принимает 0 в случае чтения и 1 в случае записи, `locality` – уровень локальности доступа 0 указывает на низкую локальность (кэш L1), 1 на среднюю локальность (кэш L2), 2 на высокую локальность (кэш L3).

Использование локальных переменных: Переменные, которые активно используются внутри циклов или функций, могут быть помещены в локальные переменные, что может улучшить вероятность попадания этих переменных в регистры или малые кэши процессора, ускоряя доступ к данным.

Оптимизация алгоритмов: Оптимизация алгоритмов может помочь уменьшить нагрузку на кэш-память, уменьшая количество доступов к данным или минимизируя переупорядочивание чтений/записей.

Некоторые алгоритмы могут работать медленней из-за нарушения пространственной или временной локальности. В качестве примера можно привести перемножение матриц. Матрица может быть локализована в памяти в виде строк, лежащих последовательно в адресном пространстве. При перемножении используется строка первой матрицы и столбец второй. Столбец не обладает пространственной локальностью и так как матрица может быть достаточно большой, можно сказать, что не обладает и временной локальностью. В связи с этим наиболее эффективно будет осуществляться умножение в случае, если для второй матрицы заранее переместить столбцы в строки.

```

#define ROWS 1024
#define COLS 1024

void multiplyMatrix(int *mat1, int *mat2, int *res) {
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            res[i * COLS + j] = 0;
            for (int k = 0; k < COLS; k++) {
                res[i * COLS + j] += mat1[i * COLS + k] * mat2[k * COLS +
j];
            }
        }
    }
    ...

void multiplyMatrixOptimized(int *mat1, int *matT2, int *res) {
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            res[i * COLS + j] = 0;
            for (int k = 0; k < COLS; k++) {
                res[i * COLS + j] += mat1[i * COLS + k] * matT2[j * COLS +
k];
            }
        }
    }
    ...

void transposeMatrix(int *mat, int *transposedMat) {
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            transposedMat[j * ROWS + i] = mat[i * COLS + j];
        }
    }
    ...

```

Время выполнения multiplyMatrix(mat1, mat2, resultSimple): 26.8099 сек

Время выполнения transposeMatrix(mat2, matT2): 0.0184 сек

Время выполнения multiplyMatrixOptimized(mat1, matT2, resultOptimize): 6.0269 сек

Тоже самое касается итераций по массиву и связанному списку. В общем случае алгоритмически сложность у этих двух структур данных будет $O(n)$, однако итерации по сплошной памяти является более выгодным, так как соседние ячейки обладают пространственной локальностью. Для связанного списка характерно, что ноды выделяются каждый раз отдельно, каждая нода указывает на соседнюю, что скорее всего приведёт к обилию кеш промахов и снижению производительности.

Кроме того часто можно видеть, как большие структуры данных инициализируют отдельными вызовами аллокации. Это отрицательно сказывается на производительности:

```
typedef struct some_struct_s{
    int value0;
    int value1;
    char *string0;
    char *string1;
} some_struct_t;

void some_struct_init(some_struct_t **st){
    *st = malloc(sizeof(some_struct_t));
    (*st)->value0 = 1;
    (*st)->value1 = 2;
    (*st)->string0 = malloc(sizeof(char)*256);
    (*st)->string1 = malloc(sizeof(char)*256);
    ...
}
```

При таком подходе не исключено, что производительность будет страдать как раз из-за отсутствия пространственной локальности.

Гораздо эффективней будет положить данные рядом, с тем чтобы восстановить пространственную локальность.

```
typedef struct some_struct_s{
    int value0;
    int value1;
    char *string0;
    char *string1;
} some_struct_t;

void some_struct_init(some_struct_t **st){
    *st = malloc(sizeof(some_struct_t) + 512);
    (*st)->value0 = 1;
    (*st)->value1 = 2;
    (*st)->string0 = ((char*)*st) +
sizeof(some_struct_t) + 0;
    (*st)->string1 = ((char*)*st) +
sizeof(some_struct_t) + 256;
```

...

Появление кешей позволило процессорам работать со своей, опережающей память скоростью. Однако существуют уязвимости, ставшие возможными как раз благодаря кешам. Одна из таких уязвимостей является Meltdown, уязвимость нарушающая безопасность и позволяющая получить несанкционированный доступ на чтение к привилегированной памяти. Meltdown становится возможным благодаря спекулятивному выполнению программы. Когда существует ветвление, процессор может заняться исполнением одного из направлений ветвления, а в некоторых архитектурах — даже обеих ветвей, в надежде иметь готовый результат вычислений к тому моменту, когда результат ветвления станет известен. При таком спекулятивном подходе возможно чтение из оперативной памяти без проверки прав доступа процесса к читаемым областям, результат исполнения таких ветвлений, даже если они не потребуются, остаются на некоторое время в кеш памяти, чем и пользуются злоумышленники. Также есть несколько похожих по воздействию уязвимостей, например Spectre.



Когерентность кешей. Протокол MESI.

Ядра CPU всегда взаимодействуют с оперативной памятью через кеш. Однако необходимо каким то образом разрешать ситуации, когда данные находятся в кеше одного процессора, но при этом в какой то момент они требуются также и в другом процессоре, как для чтения, так и для записи. Было придумано множество протоколов когерентности, отличающиеся алгоритмами, количеством состояний и, как следствие скоростью работы и масштабируемостью. Большинство современных протоколов когерентности представляют вариации протокола MESI.

Протокол MESI декларирует параметр состояния кеш линии, который может принимать четыре значения:

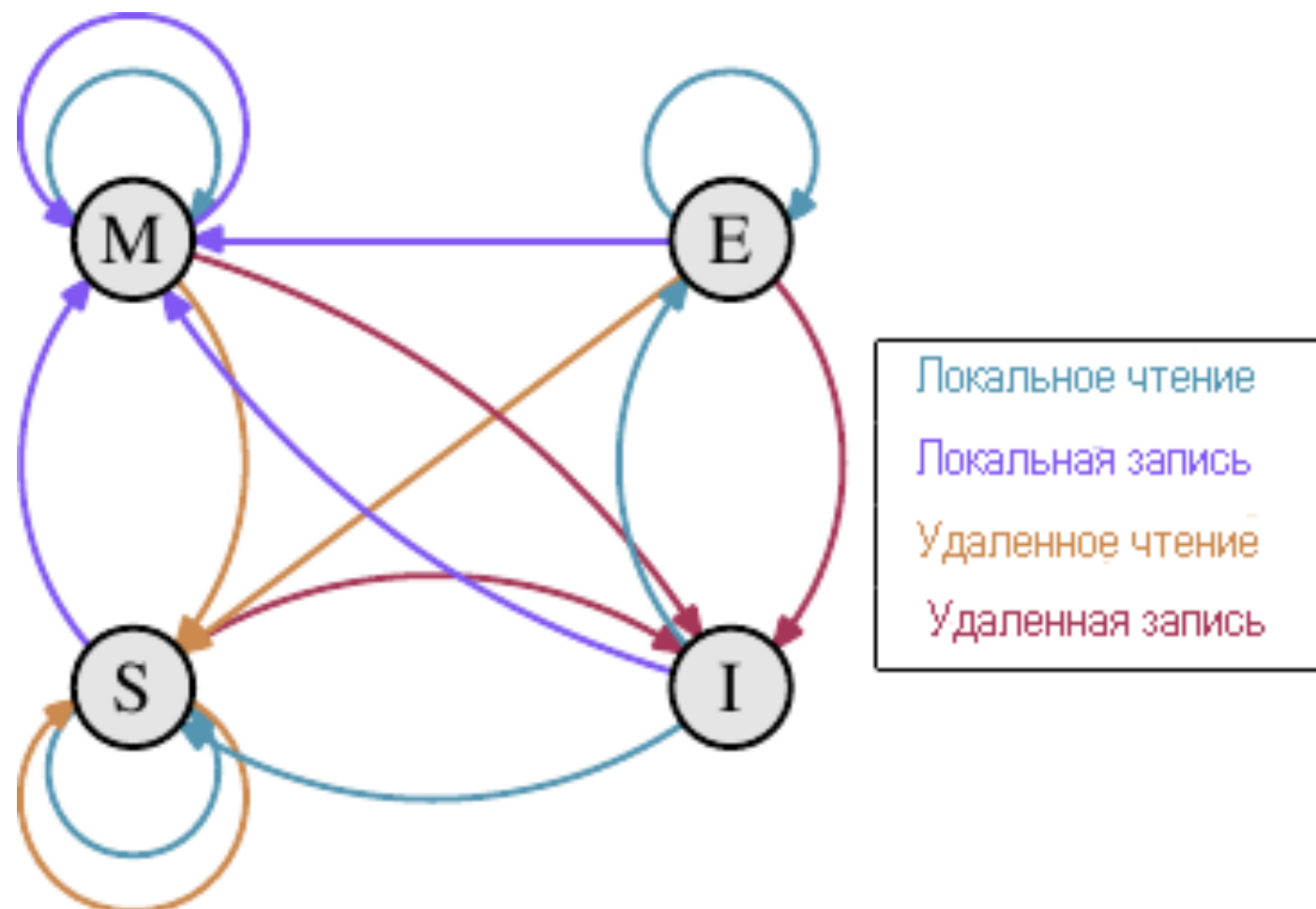
Модифицированная (M — modified). Таким флагом может быть помечена только линия в одном кэше. Данное состояние означает, что данная линия была изменена, но до памяти эти изменения еще не дошли. Хозяин такой линии может спокойно читать и писать в нее без опроса остальных.

Эксклюзивная (E — exclusive). Обозначенная таким флагом линия, так же, как и M-линия может находиться только в одном кэше. Содержащиеся в ней данные полностью идентичны данным в оперативной памяти. Записывать и читать из нее можно без внешних запросов, так как она хранится только в одном кэше. После записи такая строка должна быть помечена, как модифицированная.

Разделяемая (S — shared). Линия может одновременно содержаться в кэшах нескольких устройств и использоваться совместно. Запросы на запись в такую линию всегда идут на общую шину, что приводит к тому, что все линии с таким адресом в остальных кэшах помечаются как недействительные. При этом содержание основной памяти также обновляется. Чтение же из такой линии, не требует ни каких внешних запросов.

Недействительная (I — invalid). Такая линия считается невалидной и попытка прочитать приведет к кэш-промаху. Линия помечается недействительной в случае если она пуста или содержит устаревшую информацию.

Первоначально все кэш-строки пусты, а, следовательно, и недействительны (состояние Invalid). Если данные будут загружены для записи, то кэш-строка станет модифицированной (состояние Modified). Если данные загружаются для чтения, то новое состояние зависит от того, будет ли та же кэш-строка также загружена другим процессором. Если будет, то новое состояние будет разделяемым (состояние Shared), в противном случае — эксклюзивным (состояние Exclusive). Если кэш-строка модифицирована (состояние Modified) и чтение или запись выполняется локальным процессором, то инструкция может использовать содержимое текущего содержимое кэш-строки и её состояние не изменяется. Если второй процессор хочет читать из кэш-строки, то первый процессор должен отправить содержимое кэш-строки во второй процессор, а затем изменить состояние на разделяемое (состояние Shared).



Ядра процессора общаются благодаря специальной сущности — брокеру. Брокер имеет аппаратную реализацию, которая представляет свой определенный протокол взаимодействия. Когда одному из ядер требуется кеш линия, он отправляет запрос брокеру с пометкой read. Брокер ищет запись об этой кеш линии в соседних ядрах и если таковой нет, загружает линию из ОЗУ и помечает её как E (exclusive). Если кеш линия существует в соседних кешах, то брокер скопирует в текущее ядро найденную кеш линию и для всех ядер, в которых такая кеш линия встречается выставит флаг S (shared).

При изменении shared кеш линии, остальные процессоры уведомляются об этом с помощью брокера. Если к примеру две кеш линии в двух ядрах процессора находятся в состоянии S (shared) и одно из ядер собирается записать в линию новые данные, то первое ядро посылает специальное сообщение read/invalidate. В кеш линию будет произведена запись, далее все процессоры, которые так же владеют этой линией будут вынуждены инвалидировать её, а процессор записавший данные выставит ей тег E (exclusive).

Симулятор, позволяющий лучше понять работу протокола MESI доступен по ссылке: <https://www.scss.tcd.ie/Jeremy.Jones/VivioJS/caches/MESIHHelp.htm>

MESI Protocol

reset

bug free!

help

VivioJS help

MEMORY

address: a0 data: 6

address: a1 data: 0

address: a2 data: 0

address: a3 data: 0

CPU 0 writes 6 to a0 in memory

DATA BUS

ADDRESS BUS

BUS CYCLES: 13

SHARED

CACHE 0

CACHE 1

CACHE 2

CPU 0

CPU 1

CPU 2

read a0

write a0

read a1

write a1

read a2

write a2

read a3

write a3

read a0

write a0

read a1

write a1

read a2

write a2

read a3

write a3

read a0

write a0

read a1

write a1

read a2

write a2

read a3

write a3

E	a0	6
I		

I	a0	5
I		

I	a0	3
I		

False sharing

В некоторых случаях кажущиеся эффективными паттерны многопоточных программ могут работать медленней, чем возможно из-за кешей. Например ситуаций называемая `false sharing`.

Пример `false sharing` можно привести на достаточно простой программе. Если, например, необходимо в многопоточной программе сложить в аккумулятор значения, то чтобы синхронизировать действия всех потоков можно использовать атомарные сложения. Однако эта реализация скорее всего будет работать относительно медленно, так как при каждом вызове атомарной операции происходит синхронизация. Тогда может показаться хорошей идеей завести массив счетчиков, передать их в качестве аргументов в потоки. Потоки по мере работы могут складывать в предоставленную память данные, без использования атомарных инструкций. Если завести массив аккумуляторов, то получится ситуация, при которой несколько значений будут находиться в одной кеш линии, вызывая при этом не нужные траты на синхронизацию между CPU. Хорошую производительность покажет программа, для которой области памяти для счетчиков будут разнесены на расстояние $\geq$ размеру кеш линии.

Пример, при котором увеличение счетчика оказывается подверженным явлению `false sharing`.

```
#define COUNT 100000000  
#define THREAD_NUM 4
```

```
int thread_f(void *arg){  
    for(int i = 0; i < COUNT; ++i) *(int*)arg += 1;  
    return 0;  
}
```

...

```
int args[THREAD_NUM] = {};
```

```
for( i = 0 ; i < THREAD_NUM ; ++i ) thrd_create(&thr[i],  
thread_f, (void*)(args + i));
```

```
for( int i = 0 ; i < THREAD_NUM ; ++i ) thrd_join(thr[i], NULL);
```

```
int accumulator = 0;  
for( i = 0 ; i < THREAD_NUM ; ++i ) accumulator += *(args + i);
```

Пример без эффекта `false sharing`. В том случае данные лягут в разные кеш линии, они будут помечены `Modified` и не будут тратить лишнее процессорное время.

```
#define COUNT 100000000  
#define THREAD_NUM 4
```

```
int thread_f(void *arg){  
    for(int i = 0; i < COUNT; ++i) *(int*)arg += 1;  
    return 0;  
}
```

...

```
size_t padding = (64/sizeof(int));  
int args[THREAD_NUM * padding] __attribute__((aligned (64)));
```

```
for( i = 0 ; i < THREAD_NUM ; ++i ) thrd_create(&thr[i], thread_f,  
(void*)(args + i*padding));
```

```
for( int i = 0 ; i < THREAD_NUM ; ++i ) thrd_join(thr[i], NULL);
```

```
int accumulator = 0;  
for( i = 0 ; i < THREAD_NUM ; ++i ) accumulator += *(args + i*padding);
```


Processor affinity

Планировщик распределяет исполнение нитей на несколько исполнительных ядер исходя из определенной логики. Как правило это распределение близко к наиболее оптимальному, однако иногда требуется привязать исполнение одной нити к ядру (`processor affinity`). В Linux привязка процесса к процессору может быть установлена при помощи утилиты `taskset`. На программном уровне привязку можно инициализировать с помощью одной из функций `pthread_setaffinity_np` или `pthread_attr_setaffinity_np`.

```
#define _GNU_SOURCE
...
#include <pthread.h>

...
cpu_set_t cpuset_1;
cpu_set_t cpuset_2;

CPU_ZERO(    &cpuset_1);
CPU_SET(0,   &cpuset_1);    // привязать к ядру номер 0

CPU_ZERO(    &cpuset_2);
CPU_SET(2,   &cpuset_2);    // привязать к ядру номер 2

...
int th1_op = pthread_create(&th1, NULL, cpu_intensive_function, NULL);
int th2_op = pthread_create(&th2, NULL, cpu_intensive_function, NULL);

int s1 = pthread_setaffinity_np(th1, sizeof(cpu_set_t), &cpuset_1);
if (s1 != 0){
    errno = s1;
    perror("pthread_set_affinity_np, s1");
    exit(EXIT_FAILURE);
}

int s2 = pthread_setaffinity_np(th2, sizeof(cpu_set_t), &cpuset_2);
if (s2 != 0){
    errno = s2;
    perror("pthread_set_affinity_np, s2");
    exit(EXIT_FAILURE);
}
```

Thread Local Storage (TLS)

Использование глобальных переменных в современном программировании не рекомендуется. Некоторые старые операционные системы, такие как UNIX, изначально разрабатывались для однопроцессорного оборудования и часто используют глобальные переменные для хранения важных значений. Например `errno` — переменная, используемая для обработки ошибок многими функциями библиотеки C. На современном компьютере, где несколько потоков могут изменять `errno` переменную, вызов функции в одном потоке может перезаписать значение, ранее установленное вызовом функции в другом потоке, возможно, до того, как код в первом потоке сможет проверить наличие ошибки. Решение состоит в том, чтобы для `errno` иметь переменную, которая выглядит так, как будто она глобальная, но физически хранится в пуле памяти для каждого потока, локальном хранилище потоков.

В `posix threads` память, локальная для потока, обозначается термином `Thread-specific data`. Функции `pthread_key_create` и `pthread_key_delete` используются соответственно для создания и удаления ключа для данных, специфичных для потока. Тип ключа явно оставлен непрозрачным и обозначается как `pthread_key_t`. Этот ключ виден всем потокам. В каждом потоке ключ может быть связан с данными, относящимися к конкретному потоку, через `pthread_setspecific`. Позже данные могут быть извлечены с помощью `pthread_getspecific`.

В C11 ключевое слово `_Thread_local` используется для определения локальных переменных потока. Заголовок `<threads.h>`, если поддерживается, определяет `thread_local` как синоним этого ключевого слова.

Например:

```
#include<threads.h>
thread_local int t1 = 0;
```

Cachegrind

Одним из шагов разработки качественного программного обеспечения обычно является его оптимизация, т.е. увеличение производительности. При этом программа запускается под контролем средства профилирования, что позволяет найти места, которые требуют оптимизации. Модуль cachegrind проводит сбор статистики по попаданию в кэш первого и второго уровней процессора при выполнении операций чтения и записи данных и инструкций программ, а также статистику по работе модуля предсказания ветвлений в программах. По умолчанию, сбор статистики о предсказании ветвления инструкций (branch prediction) не проводится, и если необходимо получить эти данные, то нужно указать опцию `--branch-sim` со значением `yes`. Кроме этого, пользователь имеет возможность указания дополнительных опций, например, задающих размеры кэшей и т.п. Результаты собранные данным модулем по умолчанию выводятся в файл с именем `cachegrind.out.<pid>` (`pid` -- идентификатор процесса). Если требуется использовать другое имя файла, то можно воспользоваться опцией `--cachegrind-out-file`.